



Assignment - 2

Design And Analysis of Algorithms

Mradul Sonkar | 23B0980 Instructor: Rohit Gurjar

Explanation:

The code converts the constrained matrix problem into a flow network by encoding each cell, row, and column as a vertex in a graph. Each grid cell is represented by two vertices: a **cell-in** vertex and a **cell-out** vertex. The cell-in to cell-out edge is given a capacity equal to the difference between the cell's upper and lower bounds. Besides, to impose the minimum flow requirement (i.e., the lower bound of the cell), two auxiliary edges are inserted—one from the cell-in vertex to the sink with the same capacity as the lower bound and another from the source to the cell-out vertex with the same capacity. We ensure that before any other flow is taken into account, each cell has its minimum required flow.

For rows, there is a shared **row super source** and for each row there is an individual row-out vertex (with indices calculated using an offset off the cell vertices). An edge from the row super source to the row-out vertex is added with capacity equal to the excess flow available (i.e., the upper bound minus the lower bound), as well as extra edges that directly meet the lower bound demands. Likewise, each column is represented by a **col-in** vertex (placed after the row vertices) and a common **column super sink** is employed. Edges between each column's col-in vertex and column super sink also fit in the upper minus lower bound difference, whereas additional edges ensure that the required minimum flow is forced through the columns. In order to combine these pieces, the code links each row-out vertex to the cell-in vertices of the respective cells in that row, and each cell-out vertex to the col-in vertex of the respective column, via edges of infinite capacity. This allows the correct propagation of flow from rows to cells to columns while enforcing both cell bounds and row/column bounds.

Lastly, the algorithm runs in several phases. The initial phase applies the compulsory (lower bound) flows and check if such a flow possible. In further phases, residual capacities—computed from unused quantities between upper and lower bounds—are utilized to flow more. This, together with the vertex representations given and the auxiliary edges, enables the network to be reconfigured and analyzed in turn through the Edmonds-Karp algorithm, ensuring that all cell, row, and column constraints are enforced within the optimal final flow distribution.

Usage:

```
g++ 23B0980.cpp -o run
./run < input*
```